Applied Machine Learning Assignment

Sentiment Analysis and Face Alignment

[TODO: Your name**]**

Candidate number: **[TODO:** xxxxxx**]**

University of Sussex — G6061 Applied Machine Learning, Spring 2026

[TODO: Submission date**]**

1 Task 1: Sentiment Analysis

1.1 Description of methods

1.1.1 Problem framing

The task is binary sentiment classification of movie reviews, complicated by email-style spam (Enron-style business emails) evenly distributed across both labelled classes in the training and validation sets. The deployed system outputs a label in $\{0, 1, d\}$ where $d = -1$ denotes the dummy class assigned to documents the system identifies as spam.

1.1.2 Dataset and exploratory analysis

The supplied training set contains 11,503 documents (5,767 with label = 1 and 5,736 with label = 0; validation 1,397 documents split 704/693). Both splits are therefore close to class-balanced before any spam handling. Exploratory analysis confirms that spam contamination is approximately even across the two declared classes: applying a conservative regex marker (presence of a `Subject:` header, the token *enron*, or *unsubscribe*) flags 25.7% of declared-negative and 26.0% of declared-positive training documents, while a movie-vocabulary marker (*film*, *movie*, *director*, *plot*) fires on 22.8% and 21.0% respectively. The token-length distribution is sharply bimodal: review-style documents have a median of 21 tokens, whereas a long tail extends to 5,394 tokens for the longest email. The raw vocabulary contains 27,066 lowercased a–z token types (581,684 tokens in total), of which 55.1% appear at least twice and 28.6% at least five times — motivating a minimum document-frequency cut-off in the BoW representations. A frequency-difference analysis (greatest Δ between class rates; W02_L04) recovers expected sentiment words (*great*, *funny*, *love*, *bad*, *boring*) alongside spam-vocabulary leakage (*enron*, *hou*, *ect*, *com*, *gas*) in both class word-lists, demonstrating directly why a naive word-list or BoW classifier trained on the corrupt corpus will learn a mixture of sentiment and review-vs-spam discrimination rather than sentiment alone.

1.1.3 Pre-processing

The exploratory phase fixed two preprocessing choices made on the basis of observed data characteristics. First, tokens are obtained with a simple `[a-zA-Z']+` regex over a lowercased copy of the text, which discards punctuation and digits and matches NLTK's standard treatment of contractions (W02_L03) [?]. Second, the default NLTK English stopword list contains the negation tokens *not*, *no* and *never*, all of which carry sentiment polarity for this task; the final pipeline therefore uses a *reduced* stopword list that retains negations, as flagged in the lecture material and as a known tripwire in the assignment brief. **[TODO:** Finalise and justify remaining choices once `02_baseline` is implemented: digit handling (<NUM> placeholder vs. deletion), treatment of `!` / `?`, and the stemming [?] versus lemmatisation comparison.**]**

Figure 1: Pre-processing pipeline applied to all training, validation and test documents. **[TODO:** Replace placeholder with real flowchart and rewrite this caption to describe the choices, e.g. "Negation tokens retained; lemmatisation preferred over stemming following the trade-off discussed in Section X."**]**

1.1.4 Features and representations

[TODO: Describe the at-least-two representations you compared, e.g. TF–IDF versus averaged Word2Vec embeddings [? ?]. Explain how each was computed and why.**]**

The TF–IDF score for term t in document d is

$$\mathrm{tfidf}(d, t) = \mathrm{tf}(d, t)\, \log\left(\frac{N}{\mathrm{df}(t)}\right), \tag{1}$$

where N is the corpus size and $\mathrm{df}(t)$ is the number of documents containing t [?].

1.1.5 Spam handling

[TODO: Describe your spam-filtering stage. The lectures support cosine-similarity-to-class-centroid filtering and word-list-based filtering; both are defensible. State the threshold and how you chose it.**]**

The cosine similarity between two vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ is

$$\mathrm{sim}(\boldsymbol{a}, \boldsymbol{b}) = \frac{\boldsymbol{a} \cdot \boldsymbol{b}}{\sqrt{(\boldsymbol{a} \cdot \boldsymbol{a})(\boldsymbol{b} \cdot \boldsymbol{b})}}, \tag{2}$$

which we use to compare a candidate document against the centroids of the declared positive and negative training classes.

1.1.6 Sentiment models

[TODO: Describe each model you tried: word-list baseline, Naive Bayes [**?**], Logistic Regression, and (if used) RNN [**?**]. For each: ML task type, model family, loss function, hyperparameters and how chosen.**]**

For a binary Logistic Regression classifier with weight vector $\boldsymbol{w}$, the probability of the positive class is

$$P(y{=}1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x}) = \frac{1}{1 + \exp(-\boldsymbol{w}^\top \boldsymbol{x})}, \tag{3}$$

and parameters are obtained by minimising the cross-entropy loss.

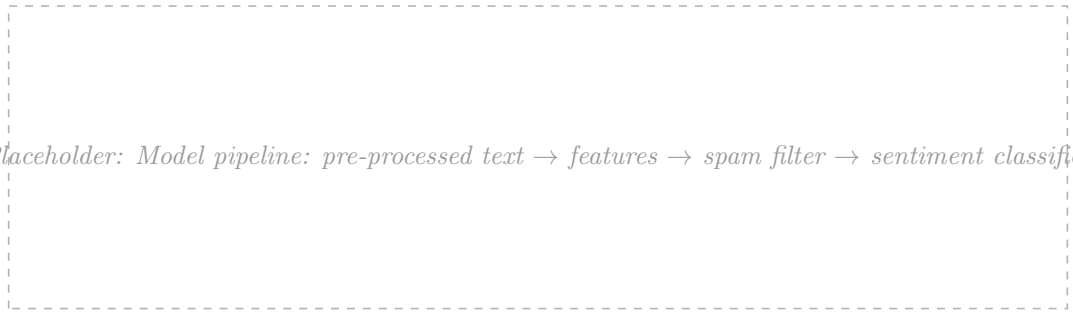


Figure 2: End-to-end sentiment system. **[TODO:** Rewrite caption to describe your final system at a high level.**]**

1.1.7 Hyperparameters and compute

Table 1: Hyperparameters and how chosen for each Task 1 approach.

Approach	Hyperparameter	Chosen value (method)
Word-list baseline	List size, margin δ	**[TODO:]**
TF–IDF + Naive Bayes	N-gram range, smoothing α	**[TODO:]**
TF–IDF + Logistic Regression	Regularisation C	**[TODO:]**
Spam threshold	Cosine cut-off	**[TODO:** (swept on validation)**]**

[TODO: One short paragraph on compute: hardware (Colab T4 / your laptop), training time per model, peak memory if relevant.**]**

1.2 Analysis of results

1.2.1 Quantitative evaluation

(a) [**TODO:** Approach name] **(b)** [**TODO:** Approach name] **(c)** [**TODO:** Approach name]

Figure 3: Confusion matrices on the held-out validation set. [**TODO:** Write a sentence or two pointing the reader at the diagonal, the spam row, and the most common confusion.]

Table 2: Validation-set performance across approaches. Precision, recall and F1 are macro-averaged across the three classes.

Approach	Accuracy	Precision	Recall	F1
Word-list baseline (no spam handling)	[TODO:]	[TODO:]	[TODO:]	[TODO:]
TF–IDF + NB (no spam handling)	[TODO:]	[TODO:]	[TODO:]	[TODO:]
TF–IDF + LR + cosine spam filter	[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO: Word2Vec + LR + cosine spam]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

[**TODO:** Discuss the precision–recall trade-off on the spam threshold and include the curve.]

1.2.2 Qualitative evaluation and failure cases

[**TODO:** Pick three to five interesting validation-set errors. For each, quote a short fragment and explain in one or two sentences *why* the model failed. Look for: negations, sarcasm, mixed sentiment, genre vocabulary used descriptively, very short reviews, spam written in a movie-review style.]

1.2.3 Systematic bias

[**TODO:** Identify and discuss systematic biases observed: e.g. length bias on the spam filter, vocabulary bias on the sentiment classifier, class imbalance effects after spam removal.]

1.3 Performance on the NLTK external dataset

[**TODO:** Report sentiment accuracy and (importantly) how often the spam detector fires on `nltk.corpus.movie_reviews`, which contains no spam. Discuss generalisation: vocabulary drift, domain shift, label-noise differences.]

2 Task 2: Face Alignment

2.1 Description of methods

2.1.1 Problem framing

The task is to predict the (x, y) coordinates of five facial landmarks (left eye, right eye, nose, left mouth corner, right mouth corner; indices 0–4) for each face image. The training data exhibit **[TODO:** briefly describe the variability you observed: rotations, brightness changes, noise, etc.**]**

2.1.2 Pre-processing

[TODO: Describe each step: resizing (and the matching transformation of the landmark coordinates), grayscale conversion, intensity scaling, optional histogram equalisation **[?].]**

Figure 4: Image pre-processing pipeline. Landmark coordinates are transformed identically so they remain consistent with the resized image geometry.

2.1.3 Features and representations

[TODO: Describe the representations you compared: raw / flattened pixels, HOG descriptors **[?**], and CNN-learned features.**]**

A brief reminder of HOG: the image region is broken into overlapping 6×6 cells, each described by a histogram of gradient orientations binned over 0–180°; blocks of 3×3 cells are then concatenated and locally normalised, yielding good invariance to brightness changes and small rotations.

2.1.4 Approaches compared

Baseline 0 — mean face. Predict the per-landmark mean of the training set for every test image. This is the zero-component PCA shape model.

Approach 1 — HOG features + linear regression. [TODO: Describe.**]**

Approach 2 — Cascaded regression with pose-indexed features [? ?]. [TODO: Describe the iterative refinement: at each stage, recompute HOG patches at the current landmark estimates and regress the residual.**]**

Approach 3 — CNN direct coordinate regression. [TODO: Describe the architecture (conv blocks, kernel size, pooling, fully-connected head). State that the loss is the squared Euclidean distance $\|\boldsymbol{h}(\boldsymbol{x}) - \boldsymbol{y}\|^2$, and the optimiser used.**]**

[TODO: Optionally: a fourth approach using heatmap regression and soft-argmax, with the hourglass-style architecture **[?].]**

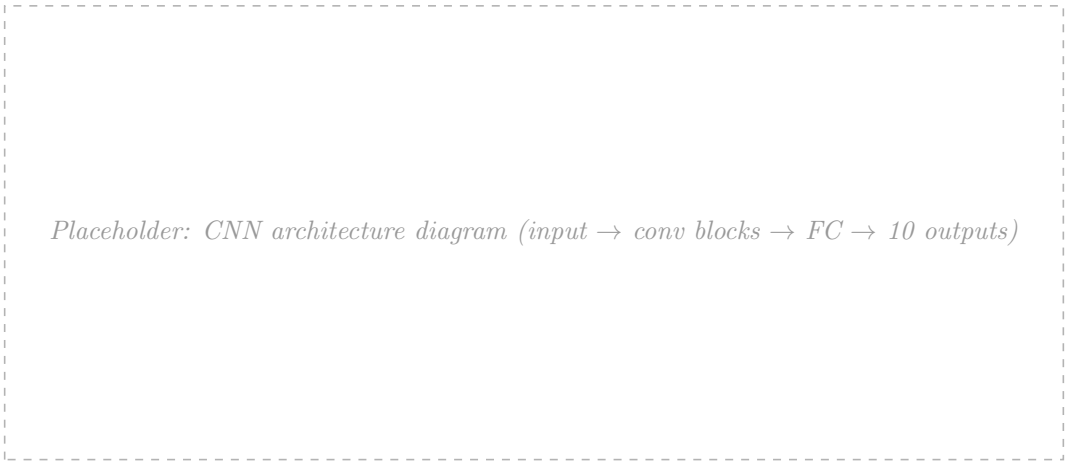


Figure 5: Architecture of the convolutional network used for direct coordinate regression. **[TODO:** Replace with your own diagram and state layer dimensions in the caption or in a small inline table.**]**

2.1.5 Data augmentation

[TODO: Describe the augmentations applied during training: rotations, scales, translations, brightness/contrast changes, and horizontal flips *with a corresponding swap of the left/right landmark indices* (eyes 0↔1; mouth corners 3↔4; nose unchanged).**]**

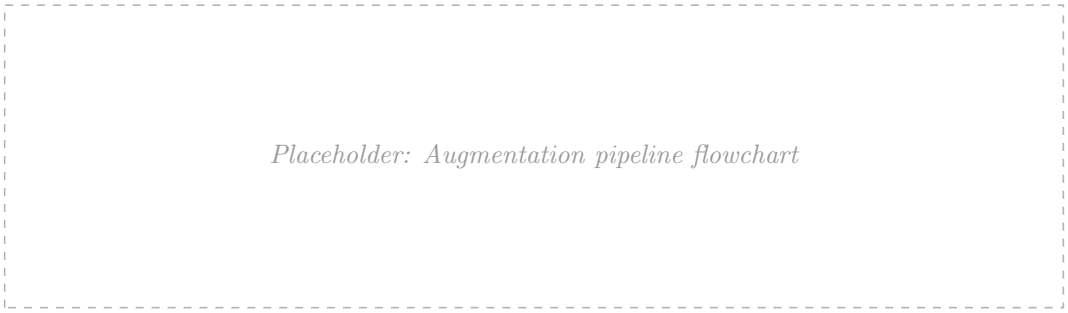


Figure 6: Data augmentation pipeline applied online during CNN training. The same geometric transform is applied to both the image and the landmark coordinates, and on horizontal flips the left/right landmark indices are swapped.

2.1.6 Hyperparameters and compute

Table 3: Hyperparameters and how chosen for each Task 2 approach.

Approach	Hyperparameter	Chosen value (method)
HOG + linear regression	Cell size, block size	**[TODO:]**
Cascaded regression	Number of stages K	**[TODO:]**
CNN direct regression	LR, batch size, epochs	**[TODO:]**

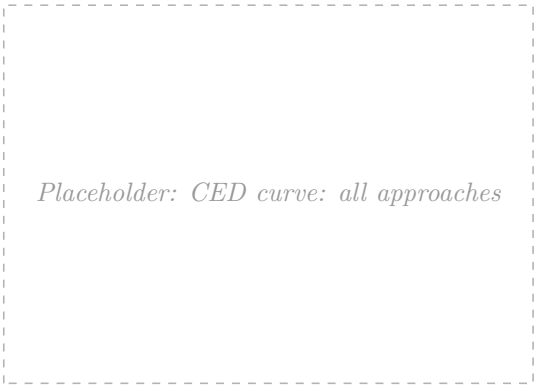
[TODO: One short paragraph: hardware (Colab T4), per-model training time.**]**

2.2 Analysis of results

The error metric is the per-landmark Euclidean distance, normalised by the inter-eye distance $d_{\text{ie}} = \|\boldsymbol{p}_0 - \boldsymbol{p}_1\|$:

$$\text{NME}_i = \frac{\|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|}{d_{\text{ie}}}, \tag{4}$$

with the per-image error reported as the mean over the five landmarks.



(a) Cumulative error distribution.

(b) Per-image NME boxplots.

Figure 7: Quantitative comparison on the held-out validation set. **[TODO:** Note that the mean face is included as the floor; comment on which approach is uniformly better and where the curves cross.**]**

Figure 8: Per-landmark mean NME for each approach. **[TODO:** Comment on which landmark is hardest and why — e.g. the nose has less local texture, or mouth corners shift with expression.**]**

2.2.1 Qualitative examples

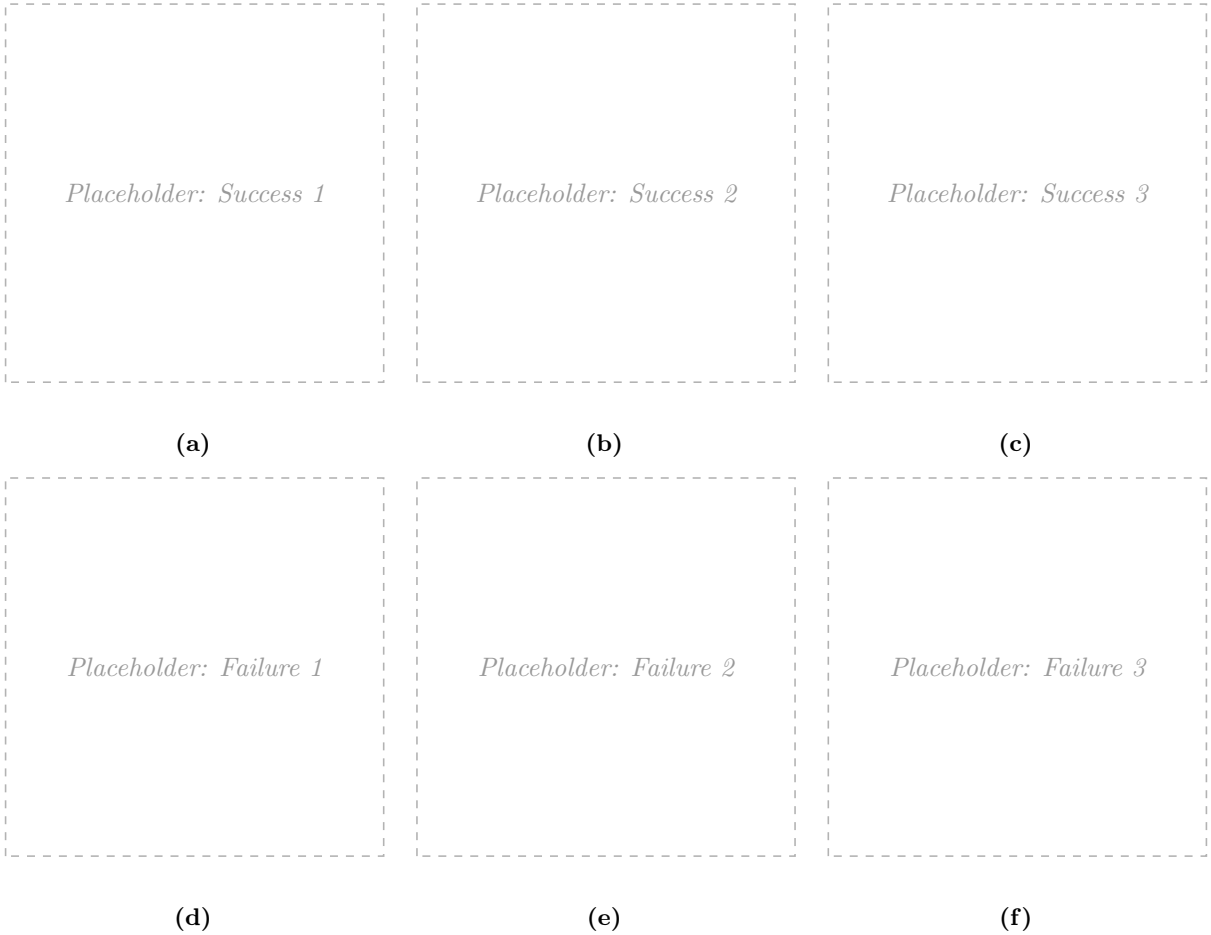
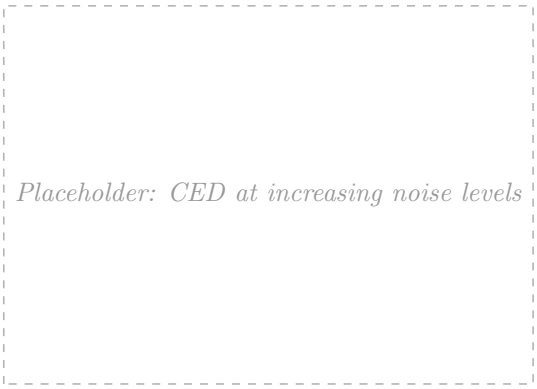


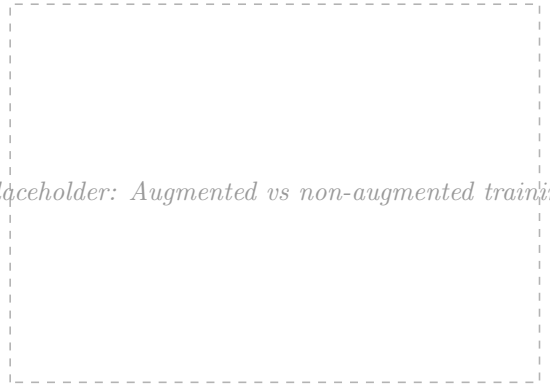
Figure 9: Predicted (red) and ground-truth (green) landmarks on representative successes (top) and failures (bottom). **[TODO:** In the caption, briefly describe what makes each failure hard.**]**

2.3 Extended robustness analysis

[TODO: Describe the perturbations applied (e.g. Gaussian noise at $\sigma \in \{0, 5, 10, 20, 40\}$ on a 0–255 scale; rotations of $\{0, 5, 10, 20, 40\}^\circ$) and which approach was tested.**]**



(a) Robustness to additive Gaussian noise.



(b) Effect of training-time augmentation.

Figure 10: Robustness analysis. **[TODO:** Discuss why the chosen approach is more robust to certain perturbations: HOG's invariance to brightness and small rotations; convolutional translation equivariance; augmentation teaching the network seen transformations.**]**

Generative AI declaration

[TODO: State concretely how generative AI was used (e.g. for ideation, debugging, or LaTeX formatting), which tool, and that you verified all output. Per the brief, generative AI was *not* used to write this report or to analyse data end-to-end.**]**

References

[] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009. `https://www.nltk.org`.

[] Xudong Cao, Yichen Wei, Fang Wen, and Jian Sun. Face alignment by explicit shape regression. *International Journal of Computer Vision*, 107(2):177–190, 2014.

[] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Online manuscript, 3rd, draft edition, 2026. `https://web.stanford.edu/~jurafsky/slp3/`.

[] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word

representations in vector space. In *International Conference on Learning Representations (ICLR), Workshop Track*, 2013.

[] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[] Alejandro Newell, Kaiyu Yang, and Jia Deng. Stacked hourglass networks for human pose estimation. In *European Conference on Computer Vision (ECCV)*, 2016.

[] Martin F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.

[] Xuehan Xiong and Fernando De la Torre. Supervised descent method and its applications to face alignment. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013.